

# AI Co-Pilot을 활용한 견고한 코드: TDD와 리팩토링

## AI Engineer 로의 관점 전환

*"AI is your Pair Programmer,  
not your replacement."*

*- The Modern Engineer's Mindset*

# 실습

## Productivity

단순 반복 코딩은 AI에게 맡기고,  
개발자는 설계와 아키텍처에 집중해 생산성을 2개로 높입니다.

## Modernization

Legacy Class 컴포넌트를 안전하게  
최신 Hooks 패턴으로 마이그레이션 합니다.

## TDD & Quality

작성하기 번거로웠던 테스트 코드를 AI로 5분 만에 생성하고,  
엣지 케이스까지 커버합니다.

# 0. 프로젝트 생성 및 환경 설정

## # 1. 프로젝트 생성 (pnpm 사용)

```
pnpm create vite react-refactoring --template react-ts  
cd react-refactoring  
pnpm install
```

## # 2. 테스트 및 유틸리티 라이브러리 설치 (Vitest, Testing Library)

```
pnpm add -D vitest jsdom @testing-library/react @testing-library/jest-dom @types/node
```

## # 3. 실습용 폴더 구조 생성

```
mkdir -p src/session1-analysis  
mkdir -p src/session2-legacy  
mkdir -p src/session3-tdd  
mkdir -p src/session4-security
```

# 1. 테스트 환경 설정 (Vitest Config)

Vite 프로젝트에서 테스트가 동작하도록 설정 파일 수정

## 1. vite.config.ts 수정

```
// vite.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: './src/setupTests.ts',
  },
});
```

## 2. src/setupTests.ts 생성

```
// src/setupTests.ts
import '@testing-library/jest-dom';
```

# Prompt Engineering: Context Injection

## ❌ Bad Prompt

"이 코드 좀 고쳐줘."

- 의도가 불분명함
- 단순 문법 오류만 수정할 가능성 높음
- 비즈니스 맥락 누락

## ✅ Good Prompt

"너는 시니어 React 엔지니어야. 이 코드를 메모리 누수 관점에서 분석하고, useEffect 의존성 배열을 최적화해서 리팩토링 해줘."

- 페르소나 부여 (시니어 엔지니어)
- 구체적 목표 제시 (메모리 누수, 최적화)

## 2. AI 마인드셋 & 코드 분석

파일 생성: src/session1-analysis/ComplexCalculation.ts

### Prompt 1

이 TypeScript 함수의 문제점을 분석하고 개선된 코드를 제안해줘

[코드 붙여넣기]

### Prompt 2

이 코드의 문제점을 단계별로 설명해줘:

Step 1: 가장 심각한 성능 문제 1가지만 찾아줘

Step 2: 그 문제를 해결하는 코드를 보여줘

Step 3: 다른 개선점들도 순차적으로 적용해줘

각 단계마다 왜 그렇게 개선하는지 설명해줘.

[코드 붙여넣기]

### 3. Legacy 리팩토링 (Class → Hooks)

파일 생성: src/ession2-legacy/EmployeeDashboardClass.tsx

#### Prompt 1

이 React 클래스 컴포넌트를 함수형 컴포넌트로 변환해줘:

요구사항:

1. useState, useEffect 혹 사용
2. 로직을 커스텀 훅(useEmployees)으로 분리
3. TypeScript 타입 유지
4. 기능은 동일하게 유지

[코드 붙여넣기]

#### Prompt 2

이 클래스 컴포넌트를 함수형 컴포넌트로 변환하는 과정을 단계별로 설명해줘:

Step 1: constructor의 state를 useState로 변환

Step 2: componentDidMount와 componentDidUpdate를 useEffect로 변환

Step 3: 비즈니스 로직을 커스텀 훅으로 분리 (useEmployees)

Step 4: 최종 리팩토링 (가독성 개선)

각 단계마다:

- 변환된 코드
- 왜 이렇게 변환하는지 설명
- 주의할 점

[코드 붙여넣기]



# AI-Driven TDD Workflow

## 1 Spec Definition (말로 하는 설계)

"비밀번호는 8자 이상, 특수문자 포함되어야 해"라고 자연어로 규칙을 정의합니다.

## 2 Test Generation (Red)

AI에게 "위 규칙을 검증할 Jest 테스트 케이스를 짜줘"라고 요청하여 실패하는 테스트를 먼저 만듭니다.

## 3 Implementation (Green)

"이 테스트를 통과하는 최소한의 코드를 작성해줘"라고 요청하여 기능을 구현합니다.

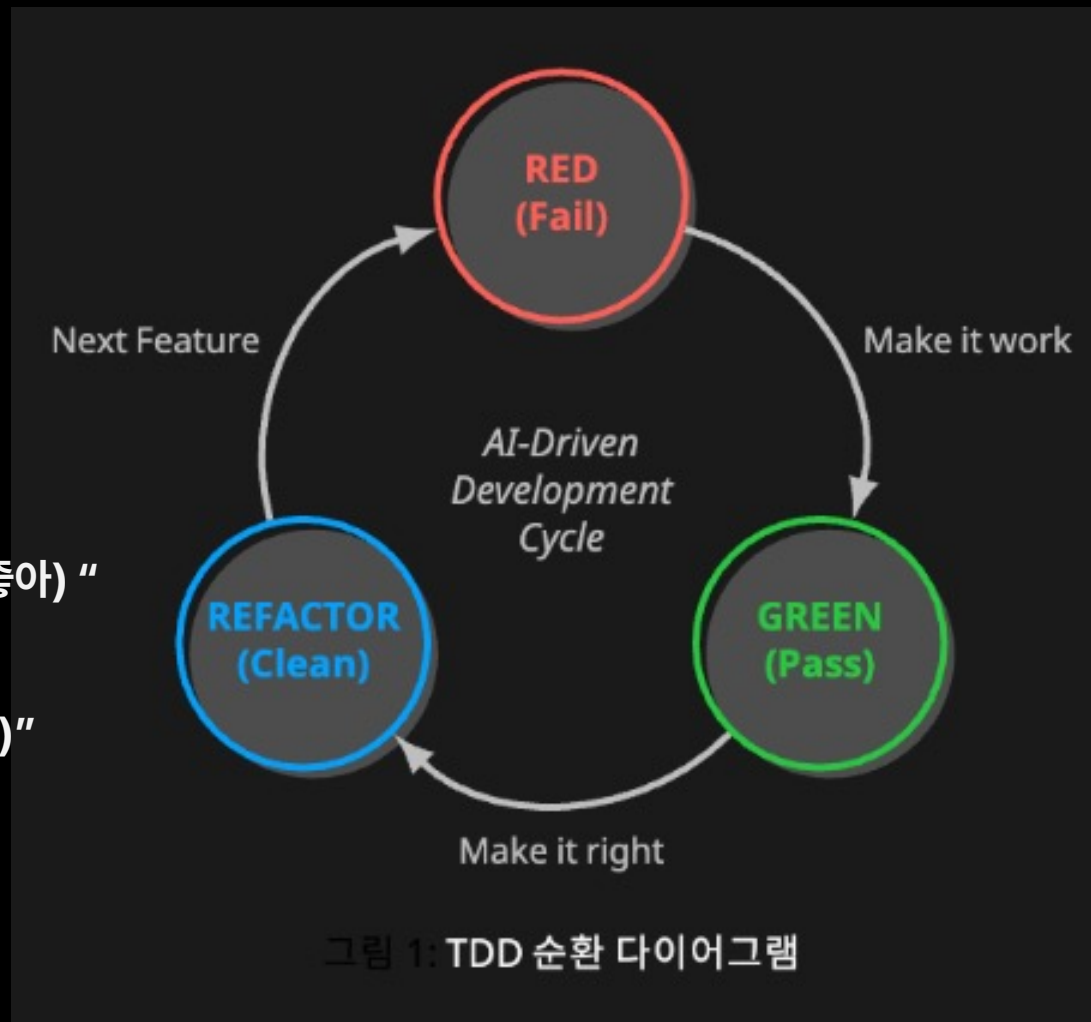
## 4 Edge Case Discovery

"내가 놓친 예외 상황은 없어?"라고 AI에게 되물어 코드의 구멍을 메웁니다.

# The TDD Cycle

## 단계별 AI 프롬프트 전략

- **Red**  
“이 기능을 검증할 수 있는 테스트 케이스를 먼저 작성해줘.  
(실패하는 것이 정상)”
- **Green**  
“테스트를 통과하는 최소한의 코드를 작성해줘. (하드코딩이어도 좋아) ”
- **Refactoring**  
“중복을 제거하고 가독성을 높여서 리팩토링 해줘. (기능은 유지해)”



## 4. AI-Driven TDD

### Prompt 1

useEmployees 커스텀 훅에 대한 단위 테스트를 Vitest로 작성해줘:

요구사항:

1. Vitest + @testing-library/react 사용
2. 다음 시나리오 테스트:
  - 초기 상태 확인
  - 부서 변경 시 데이터 재조회
  - 로딩 상태 변화
  - 에러 처리
3. Mock 데이터 사용
4. 각 테스트에 설명 주석 포함

테스트할 훅:

[useEmployees 코드 붙여넣기]

### Prompt 2

useEmployees 훅에 대한 단위 테스트와 통합 테스트를 모두 작성해줘:

단위 테스트 (useEmployees.test.ts):

- 훅의 개별 기능 테스트
- Mock 데이터 사용

통합 테스트 (EmployeeDashboard.test.tsx):

- 실제 컴포넌트와 훅의 통합
- 사용자 상호작용 시뮬레이션
- UI 렌더링 검증

Vitest 설정 파일(vitest.config.ts)도 함께 제공해줘.

[useEmployees 코드와 EmployeeDashboard 컴포넌트 붙여넣기]

# 5. Code Quality & Security

## Prompt 1

이 React 컴포넌트의 보안 취약점을 분석하고 수정해줘:

요구사항:

1. 발견된 모든 보안 취약점 나열
2. 각 취약점의 위험도 평가 (Critical/High/Medium/Low)
3. 공격 시나리오 예시
4. 안전한 코드로 수정
5. 추가 보안 권장사항

[코드 붙여넣기]

## Prompt 2

이 컴포넌트의 XSS(Cross-Site Scripting) 취약점을 심층 분석해줘:  
분석 요청사항:

1. dangerouslySetInnerHTML 사용의 문제점
2. 구체적인 XSS 공격 예시 (실제 악성 코드)
3. 공격 성공 시 발생 가능한 피해
4. 안전한 대안 3가지 제시:
  - DOMPurify 사용 ← 예시 1
  - 마크다운 라이브러리 사용 ← 예시 2
  - React의 기본 이스케이핑 활용 ← 예시 3

수정 코드 요구사항:

- TypeScript 타입 안정성 유지
- 성능 최적화 고려
- 사용자 경험 저하 없이 보안 강화
- 각 대안의 장단점 비교

[코드 붙여넣기]